

Relatório do Projeto de Ray Tracing - INF2604

1. Introdução

Este relatório detalha a implementação de um sistema de Ray Tracing em C++ desenvolvido como parte da disciplina INF2604. O objetivo principal do projeto foi renderizar cenas 3D contendo diferentes formas geométricas, iluminação e materiais, utilizando o algoritmo de traçado de raios.

O projeto implementa os requisitos básicos, incluindo a renderização de esferas e caixas com iluminação direta (modelo de Phong) e sombras projetadas por fontes de luz pontuais, além de suporte a múltiplas amostras por pixel para anti-aliasing (7pts). Adicionalmente, foram implementadas as seguintes extensões:

- Aplicação de transformações de modelagem (translação e rotação) em objetos. (1pt)
- Instanciação de fonte de luz retangular (luz de área) para geração de sombras suaves. (1pt)
- Instanciação de objetos com materiais reflexivos. (1pt)

2. Arquitetura e Componentes Principais

O projeto foi estruturado de forma modular, com os componentes principais organizados em subdiretórios dentro de `include/`:

2.1. Core (`include/core/`)

Contém as classes fundamentais para o funcionamento do ray tracer:

- `Vector3` : Representa vetores e pontos 3D, com operações matemáticas essenciais (adição, subtração, produto escalar, produto vetorial, normalização, etc.).
- `Color` : Representa cores RGB usando componentes `float` . Inclui operadores para manipulação de cores (adição, multiplicação por escalar, multiplicação

componente a componente) e conversão para formato 8-bit (0-255) para saída de imagem. É intrinsecamente ligada ao `Vector3`.

- `Ray`: Define um raio 3D com uma origem (`Vector3`) e uma direção (`Vector3`). Possui um método `pointAtParameter(t)` para calcular um ponto ao longo do raio.
- `Camera`: Modela a câmera virtual, definida por sua posição, ponto de observação (`lookAt`), vetor `up`, campo de visão (FOV), razão de aspecto e distância focal. Calcula uma base ortonormal (u, v, w) e gera raios primários (`getRay`) que partem da câmera em direção ao plano de imagem.
- `Renderer`: Orquestra o processo de renderização. Recebe as dimensões da imagem, número de amostras por pixel e profundidade máxima de recursão. O método `render` itera sobre cada pixel, gera múltiplos raios por pixel (usando amostragem estratificada para anti-aliasing), chama `traceRay` para cada raio, calcula a média das cores resultantes, aplica correção gamma e ajuste de exposição, e armazena o resultado. O método `traceRay` é recursivo e responsável por:
 - Encontrar a interseção mais próxima do raio com a cena.
 - Calcular a cor no ponto de interseção, considerando iluminação direta (`calculateDirectLight`) e contribuições recursivas (como reflexão).
 - `calculateDirectLight` calcula a iluminação ambiente e a contribuição de cada luz (usando o método `shade` do material), verificando oclusão (sombras) através de `Scene::isShadowed`.
 - A classe utiliza OpenMP (`#pragma omp`) para paralelizar o loop de renderização principal, acelerando o processo.
 - Inclui o método `saveToPPM` para salvar a imagem renderizada no formato PPM.

2.2. Geometria (`include/geometry/`)

Define as formas geométricas e a cena:

- `Primitive`: Classe base abstrata para todos os objetos renderizáveis. Define a interface `hit`, que deve ser implementada por subclasses para testar a interseção com um raio.

- **HitRecord** : Estrutura que armazena informações sobre uma interseção raio-objeto (parâmetro **t**, ponto de interseção, normal da superfície, ponteiro para o material e orientação da face). Contém **setFaceNormal** para garantir que a normal sempre aponte contra o raio incidente.
- **Sphere** : Subclasse de **Primitive** que implementa uma esfera, definida por centro, raio e material. O método **hit** resolve a equação quadrática para a interseção raio-esfera.
- **Box** : Subclasse de **Primitive** que implementa uma caixa alinhada aos eixos, definida por dois pontos (**min** e **max**) e um material. O método **hit** utiliza o algoritmo de "slab test" para verificar interseções com os planos que definem a caixa.
- **Scene** : Agrega todos os elementos da cena: uma coleção de **Primitive*** (objetos), uma coleção de **Light*** (luzes) e a **AmbientLight** . O método **hit** itera sobre os objetos para encontrar a interseção mais próxima. O método **isShadowed** verifica se um ponto está visível para uma determinada fonte de luz, lançando um raio de sombra.

2.3. Materiais (**include/material/**)

Define as propriedades de superfície dos objetos:

- **Material** : Classe base para materiais. Armazena componentes de cor ambiente, difusa e especular, além do expoente de brilho (**shininess**). O método virtual **shade** implementa o modelo de iluminação de Phong padrão, calculando as contribuições difusa (Lambertiana) e especular.
- **ReflectiveMaterial** : Subclasse de **Material** que adiciona um coeficiente de refletividade (**reflectivity**). Usada pelo **Renderer** para calcular a cor refletida: um novo raio é gerado na direção da reflexão e traçado recursivamente através de uma chamada a **traceRay** . A cor final é uma interpolação entre a cor da iluminação direta e a cor refletida, baseada na **reflectivity** .
- **EmissiveMaterial** : Subclasse de **Material** para superfícies que emitem luz (e devem ser visíveis como tal). Possui uma cor de **emission** . O método **shade** simplesmente retorna essa cor. O **Renderer** também trata colisões com esses materiais de forma especial, retornando diretamente a cor de emissão.

2.4. Luzes (`include/light/`)

Define as fontes de iluminação da cena:

- `Light` : Interface (classe base abstrata) para fontes de luz. Define métodos virtuais puros para obter a direção da luz a partir de um ponto (`getDirection`), a intensidade da luz em um ponto (`getIntensity`) e a distância até a luz (`getDistance`).
- `AmbientLight` : Representa a luz ambiente global, com uma intensidade de cor uniforme. Não herda de `Light` , sendo tratada separadamente pela `Scene` e `Renderer` .
- `PointLight` : Subclasse de `Light` que representa uma fonte de luz pontual, definida por posição e intensidade.
- `RectangularLight` : Subclasse de `Light` que representa uma fonte de luz de área retangular (definida por um canto e dois vetores, `u` e `v`). Para calcular a direção, intensidade e distância, ela utiliza um método `samplePoint` que retorna um ponto aleatório na superfície da luz a cada chamada. Isso é essencial para gerar sombras suaves (soft shadows).

2.5. Transformações (`include/transform/`)

Permite aplicar transformações geométricas aos objetos:

- `Transform` : Classe base abstrata que herda de `Primitive` . Funciona como um "wrapper" em torno de outro `Primitive` , aplicando uma transformação a ele.
- `Translate` : Subclasse de `Transform` que aplica uma translação (deslocamento) a um objeto. Armazena um vetor `offset` . O método `hit` funciona aplicando a translação *inversa* ao raio incidente antes de testar a interseção com o objeto original. O ponto de interseção resultante é então transladado de volta.
- `Rotate` : Subclasse de `Transform` que aplica uma rotação em torno de um eixo arbitrário. Armazena o ângulo e o eixo de rotação. Pré-calcula as matrizes de rotação e rotação inversa. O método `hit` aplica a rotação *inversa* ao raio (origem e direção) antes de testar a interseção com o objeto original. O ponto de interseção e a normal são então rotacionados de volta para o espaço original.

3. Implementação da Cena e Funcionalidades Adicionais

As cenas de exemplo ([examples/](#)) demonstram como os componentes descritos acima são instanciados e combinados.

3.1. Renderização Básica (Modelo Phong, Sombras, Múltiplas Amostras)

A funcionalidade básica de renderização é evidente em todos os exemplos. Uma `Scene` é populada com objetos (`Box` , `Sphere`), cada um associado a um `Material` . Fontes de luz (`PointLight` , `RectangularLight`) e uma `AmbientLight` são adicionadas. Uma `Camera` é configurada. O `Renderer::render` então calcula a cor de cada pixel:

- Lançando múltiplos raios por pixel (controlado por `samplesPerPixel`) para anti-aliasing.
- Para cada raio, `Renderer::traceRay` encontra a interseção mais próxima.
- `Renderer::calculateDirectLight` calcula a cor baseada no `Material` do objeto atingido, na `AmbientLight` da cena e na contribuição de cada `Light` .
- A contribuição de cada `Light` envolve:
 - Chamar `Material::shade` para calcular a cor Phong (difusa + especular).
 - Chamar `Scene::isShadowed` para verificar se há oclusão entre o ponto e a luz (sombras).
- Os resultados das múltiplas amostras são promediados para a cor final do pixel.

3.2. Transformações de Modelagem

As classes `Translate` e `Rotate` são usadas nos exemplos `cornell_box.cpp` e `rectangular_light_scene.cpp` para posicionar e orientar os dois blocos dentro da caixa. A instanciação segue um padrão de encapsulamento:

```
// Cria a geometria base
Box* baseBox = new Box(/*...*/);
// Aplica a rotação
Rotate* rotatedBox = new Rotate(baseBox, angle, axis);
```

```
// Aplica a translação ao objeto já rotacionado
Translate* finalBox = new Translate(rotatedBox, offset);
// Adiciona o objeto transformado à cena
scene.addObject(finalBox);
```

O ray tracer lida com isso aplicando as transformações inversas ao raio de forma aninhada durante o teste de interseção.

3.3. Fonte de Luz Retangular

A classe `RectangularLight` é usada em `rectangular_light_scene.cpp` para substituir a `PointLight` da cena `cornell_box.cpp`. A principal diferença é que `RectangularLight::getDirection`, `getIntensity` e `getDistance` amostram um ponto aleatório na superfície da luz a cada chamada. Quando `Renderer::calculateDirectLight` e `Scene::isShadowed` usam esses métodos (especialmente `isShadowed`, que lança múltiplos raios de sombra devido às múltiplas amostras por pixel), diferentes pontos na luz são considerados, resultando em penumbras e sombras mais suaves em comparação com as sombras "duras" (hard shadows) geradas pela `PointLight`. A cena `rectangular_light_scene.cpp` também instancia um `Box` fino e visível na posição da luz para representá-la geometricamente. Geralmente, luzes de área exigem um número maior de `samplesPerPixel` para reduzir o ruído nas sombras.

3.4. Objetos Reflexivos

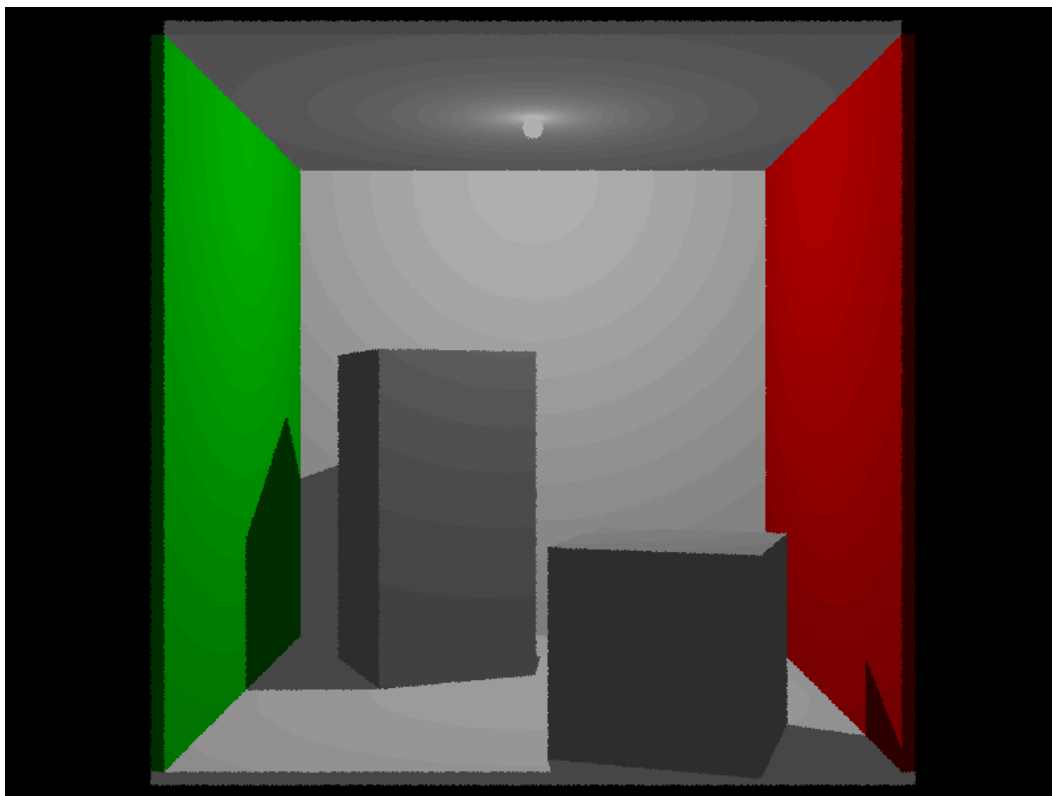
A classe `ReflectiveMaterial` permite simular superfícies espelhadas ou parcialmente reflexivas. Ela herda de `Material` e adiciona o parâmetro `reflectivity` (0.0 a 1.0). Quando `Renderer::traceRay` detecta uma colisão com um `ReflectiveMaterial`, além de calcular a iluminação direta (`calculateDirectLight`), ele também chama `ReflectiveMaterial::calculateReflection`. Esta função calcula a direção do raio refletido e invoca a função `traceRay` original recursivamente para esse novo raio (incrementando a profundidade `depth`). A cor final retornada por `traceRay` para um material reflexivo é a combinação da cor da iluminação direta (ponderada por `1.0 - reflectivity`) e da cor obtida pelo raio refletido (ponderada por `reflectivity`). A profundidade máxima de recursão (`maxDepth` no `Renderer`) limita o número de reflexões calculadas. Esta funcionalidade é demonstrada no exemplo `reflective_scene.cpp`.

4. Resultados Visuais

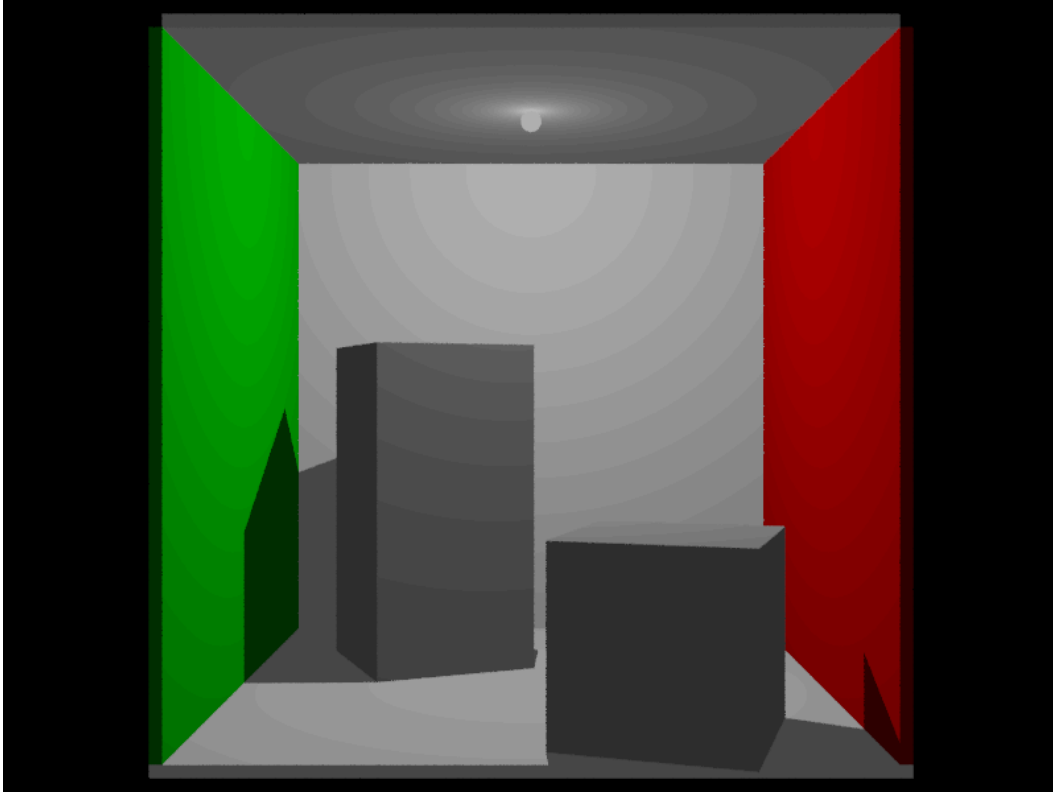
4.1. Efeito do Número de Amostras por Pixel (Anti-aliasing)

Para analisar o efeito do anti-aliasing, a cena da Cornell Box (utilizando `PointLight`) foi renderizada com diferentes valores para `samplesPerPixel`: 1, 5, 25 e 100. As imagens resultantes (`cornell_box_1samples.png` , `cornell_box_5samples.png` , `cornell_box_25samples.png` , `cornell_box_100samples.png`) demonstram o impacto da amostragem na qualidade visual.

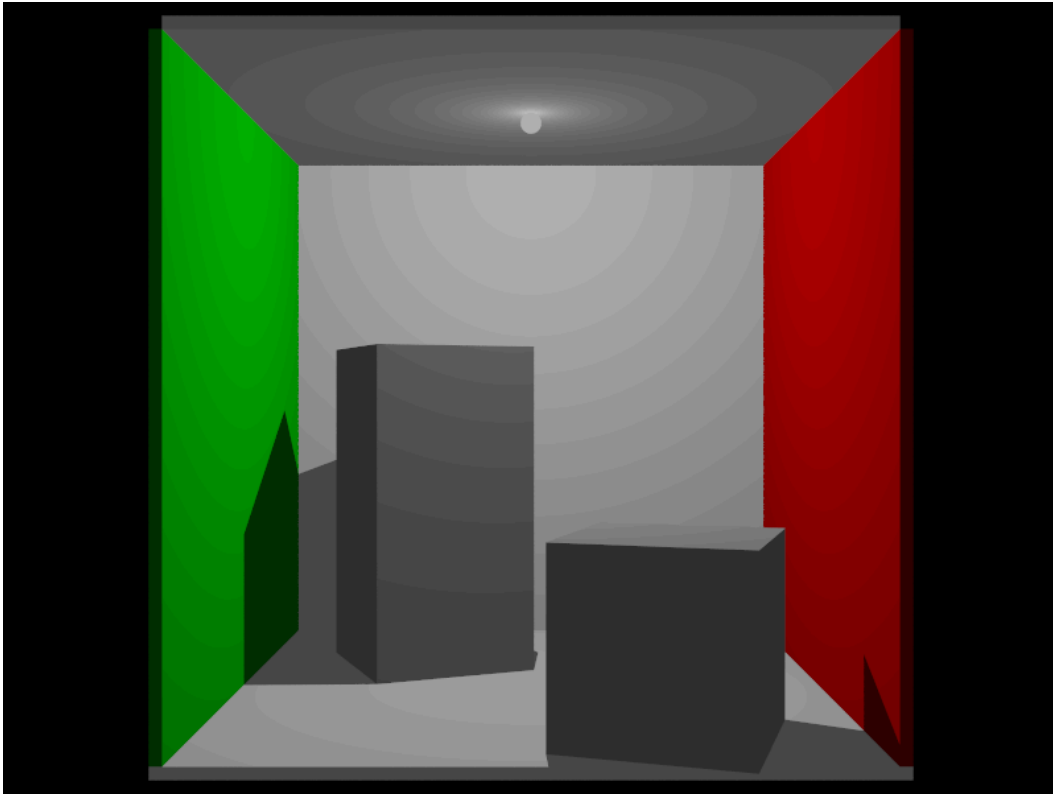
- **1 Amostra por Pixel:** Apresenta forte aliasing (serrilhado) nas bordas dos objetos e das sombras. A falta de anti-aliasing resulta em uma imagem com aparência pixelizada e artificial.



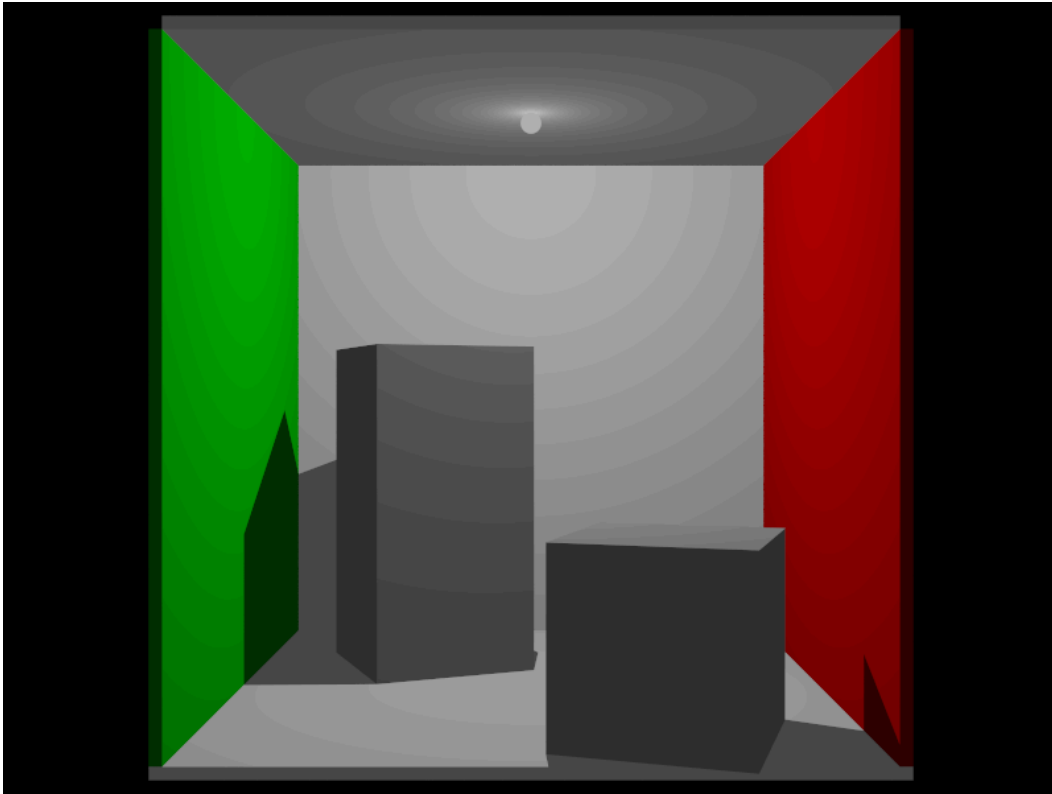
- **5 Amostras por Pixel:** Mostra uma melhora notável, com redução do serrilhado, embora ainda seja visível em áreas de alto contraste.



- **25 Amostras por Pixel:** Oferece um bom nível de anti-aliasing. As bordas estão significativamente mais suaves, proporcionando uma imagem com qualidade visual consideravelmente superior às anteriores.



- **100 Amostras por Pixel:** Resulta em bordas extremamente suaves e naturais. A diferença em relação a 25 amostras é mais sutil, mas perceptível, refinando ainda mais a imagem.



Conclui-se que um número maior de amostras por pixel melhora significativamente a qualidade da imagem ao reduzir o aliasing, sendo essencial para resultados visualmente agradáveis. A escolha do número de amostras envolve um compromisso entre a qualidade desejada e o tempo de processamento disponível.

4.2. Comparação: Point Light vs. Rectangular Light (Sombras Suaves)

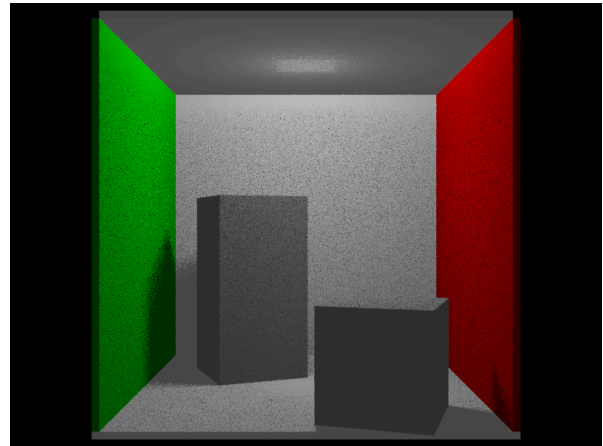
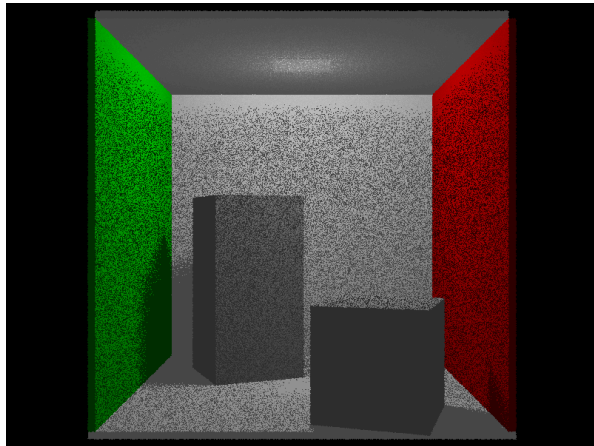
A substituição da `PointLight` por uma `RectangularLight` (luz de área) na cena da Cornell Box (`rectangular_light_scene.cpp`) introduz sombras suaves (soft shadows), que são visualmente mais realistas do que as sombras duras (hard shadows) geradas pela luz pontual.

Isso ocorre porque a `RectangularLight` emite luz de uma área finita. Para determinar a iluminação e a sombra em um ponto, o renderizador amostra pontos aleatórios na superfície da luz. A visibilidade de diferentes partes da fonte de luz a partir do ponto sombreado cria a região de penumbra (transição suave) característica das sombras suaves.

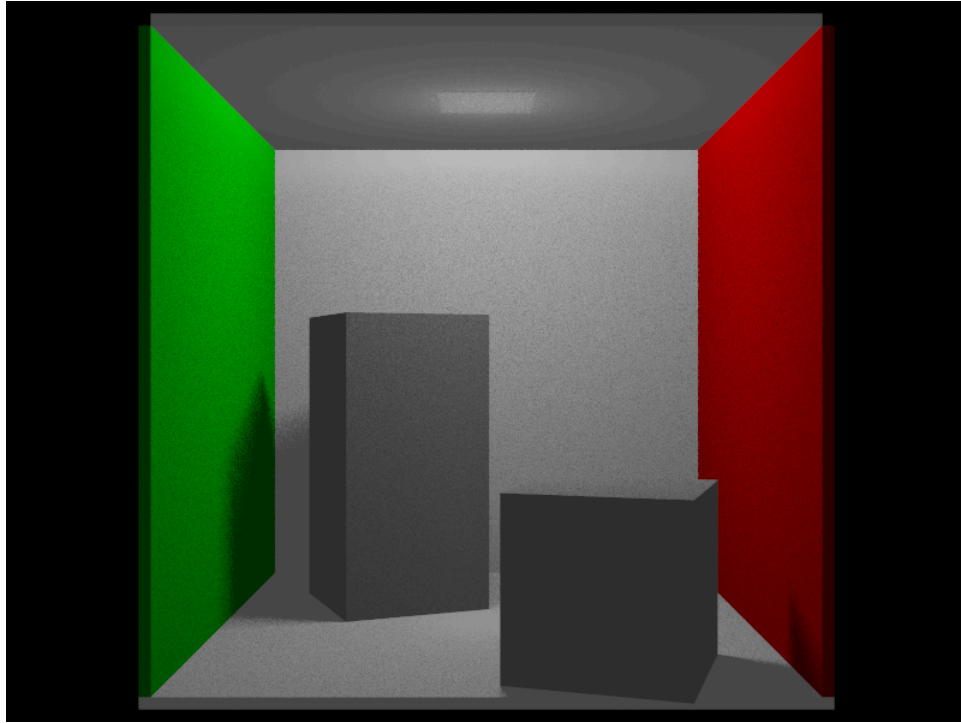
No entanto, a qualidade dessas sombras suaves é altamente dependente do número de amostras por pixel, como visto nas imagens

`rectangular_light_scene_1samples.png` , `rectangular_light_scene_5samples.png` ,
`rectangular_light_scene_25samples.png` e `rectangular_light_scene_100samples.png` :

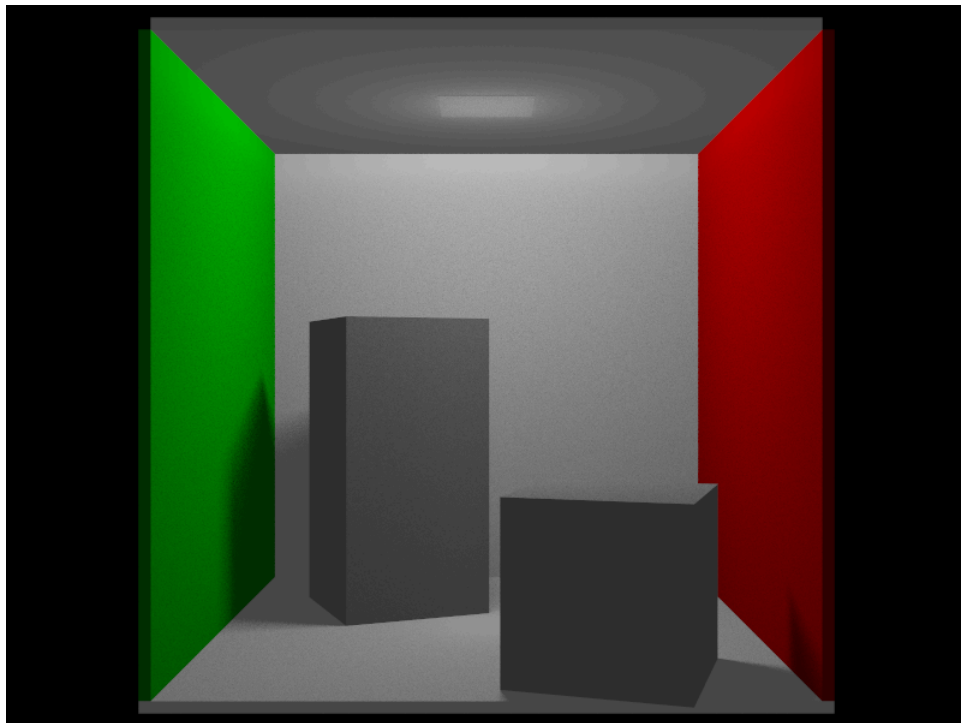
- **1 e 5 Amostras:** As sombras aparecem extremamente ruidosas e granuladas, tornando o efeito de suavidade quase irreconhecível.



- **25 Amostras:** O ruído é bastante reduzido e a penumbra se torna visível, mas a granulação ainda é aparente nas áreas de sombra.



- **100 Amostras:** As sombras suaves ficam bem definidas e com ruído mínimo, demonstrando o efeito desejado da luz de área.



Comparando `cornell_box_100samples.png` (Point Light) com `rectangular_light_scene_100samples.png` (Rectangular Light), observa-se claramente a diferença entre as sombras duras e suaves. Conclui-se que, embora as luzes de área produzam sombras mais realistas, elas exigem um número significativamente maior de amostras por pixel (e, conseqüentemente, maior tempo de renderização) para alcançar um resultado limpo e sem ruído em comparação com as luzes pontuais.

4.3. Objetos Reflexivos

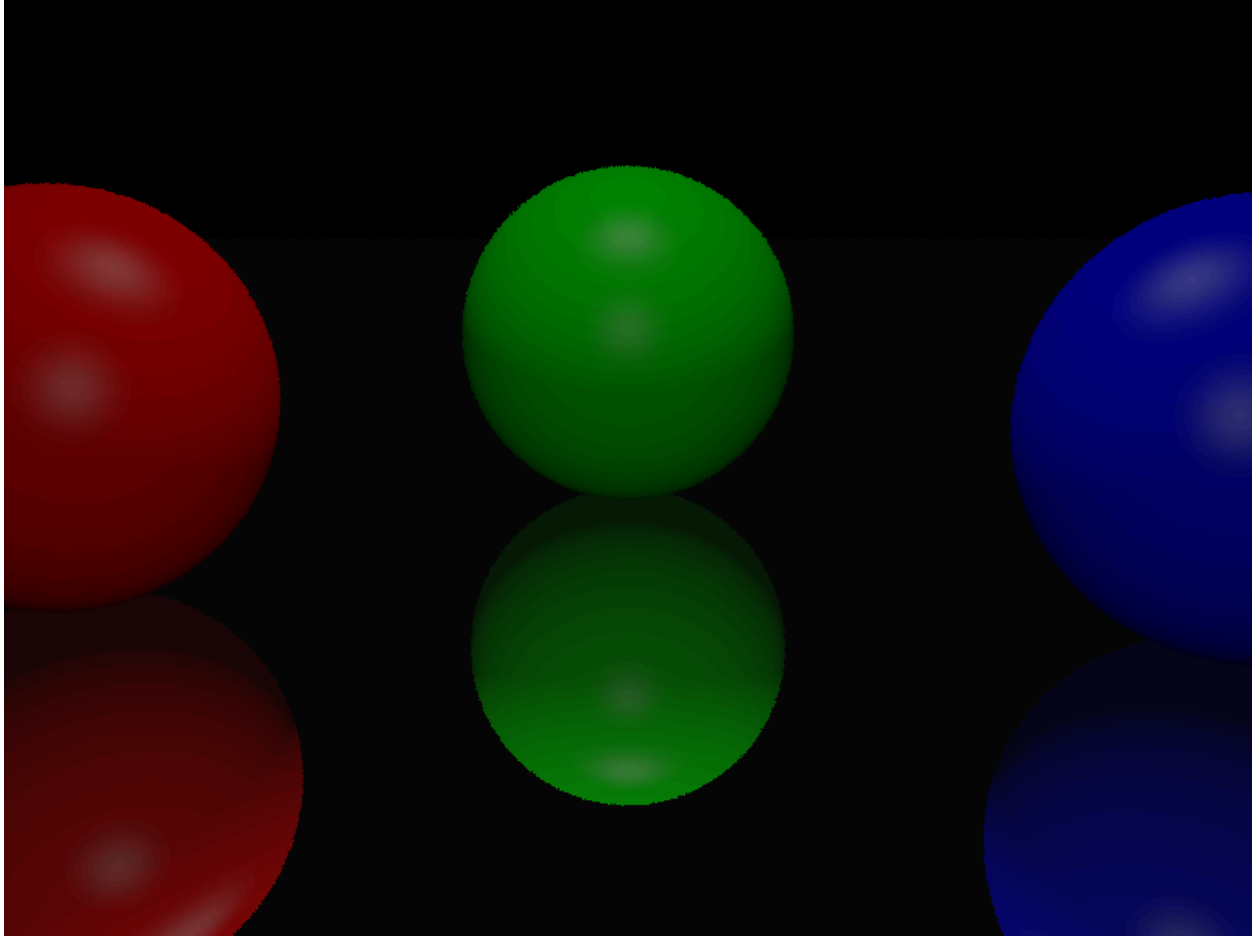
A funcionalidade de reflexão é demonstrada na cena `reflective_floor`, que utiliza um `ReflectiveMaterial` para o chão, criando um efeito de espelho. As imagens `reflective_floor_1samples.png`, `reflective_floor_5samples.png`, `reflective_floor_25samples.png` e `reflective_floor_100samples.png` ilustram o efeito.

A implementação de reflexão no `Renderer` funciona traçando raios recursivamente. Quando um raio atinge uma superfície com `ReflectiveMaterial`, um novo raio é lançado na direção da reflexão perfeita (calculada pela lei da reflexão) a partir do ponto de colisão. A cor final é uma combinação da cor da iluminação direta no ponto e da cor obtida pelo raio refletido, ponderada pela `reflectivity` do material.

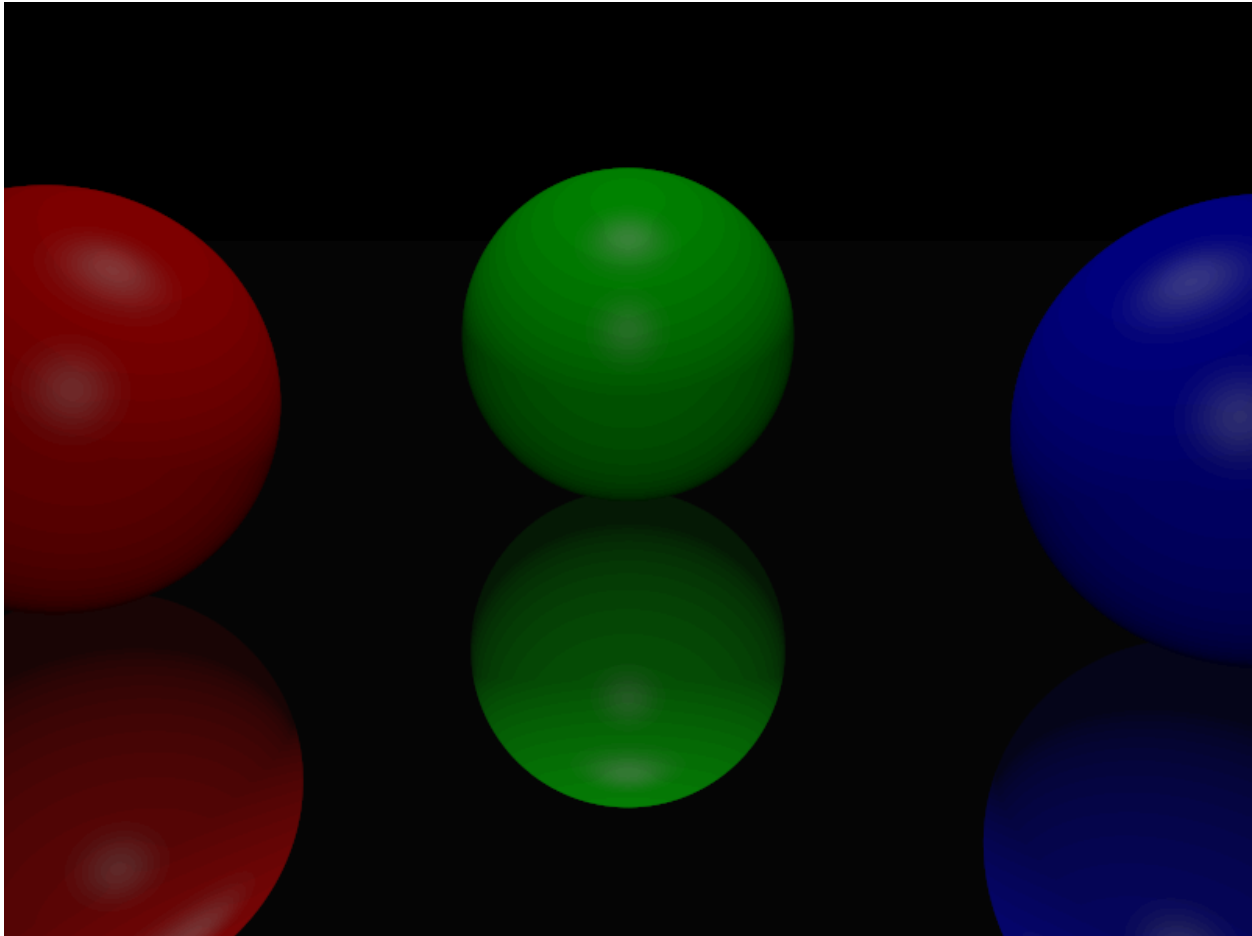
Analisando as imagens com diferentes níveis de amostragem, observa-se que:

- O principal benefício do aumento do número de amostras por pixel (SPP) nesta cena é o **anti-aliasing**, suavizando as bordas serrilhadas tanto das esferas quanto de seus reflexos no chão.
- Com 1 SPP, o aliasing é severo. Com 5 e 25 SPP, a suavidade das bordas melhora progressivamente. Com 100 SPP, as bordas ficam muito bem definidas.
- A qualidade da reflexão em si (a nitidez da imagem refletida, ignorando as bordas) não muda drasticamente com o sampling, assumindo uma reflexão especular perfeita. Não há o ruído de alta frequência que caracterizou as sombras suaves da luz retangular com poucas amostras.

1 SPP



100 SPP



Isso confirma a observação de que a melhoria visual com o aumento do sampling, embora presente e importante para a qualidade final (anti-aliasing), parece menos dramática aqui do que na cena com luz retangular. A razão é que o sampling nesta cena combate principalmente o aliasing de borda, enquanto na cena com luz retangular, combatia também (e de forma mais evidente) o ruído de alta variância introduzido pela amostragem insuficiente da área da fonte de luz.

4.4. Transformações de Modelagem

As transformações de modelagem (`Translate` e `Rotate`) foram implementadas e utilizadas para posicionar e orientar objetos na cena, como demonstrado pelos dois blocos cinza nas imagens da Cornell Box (por exemplo, `cornell_box_100samples.png` e `rectangular_light_scene_100samples.png`).

Conforme visto no código dos exemplos, cada bloco foi inicialmente criado como um `Box` na origem. Em seguida, uma transformação `Rotate` foi aplicada para girá-lo em torno do eixo Y. Finalmente, uma transformação `Translate` foi aplicada ao objeto

já rotacionado para movê-lo à sua posição final dentro da caixa. Este processo de aninhamento (`Translate(Rotate(Box))`) permite construir posicionamentos e orientações complexas.

As imagens renderizadas mostram os blocos corretamente posicionados e rotacionados, validando a implementação das classes de transformação e a capacidade do sistema de lidar com objetos transformados durante o processo de traçado de raios (através da aplicação das transformações inversas ao raio).

- Imagem da Cornell Box com Point Light.
- Imagem da Cornell Box com Rectangular Light (comparação de sombras).
- Imagens com diferentes números de amostras por pixel (demonstrando anti-aliasing e suavização de sombras da luz retangular).
- Imagem da cena com objetos reflexivos.
- Imagens mostrando o efeito das transformações nos objetos.

5. Conclusão

O projeto implementou com sucesso um ray tracer capaz de renderizar cenas 3D com iluminação Phong, sombras, anti-aliasing, transformações geométricas, luzes de área e materiais reflexivos. A arquitetura modular facilitou a adição das funcionalidades extras sobre a base de renderização. Os resultados visuais demonstram a correção das implementações, como as sombras suaves da luz retangular e os reflexos em superfícies espelhadas.